Practical No: 01

Aim: Introduction to TensorFlow

a. Create tensors with different shapes and data types.
b. Perform basic operations like addition, subtraction, multiplication, and division on tensors.
c. Reshape, slice, and index tensors to extract specific elements or sections.
d. Performing matrix multiplication and finding eigenvectors and eigenvalues using TensorFlow

Description:

II) Program to solve the XOR problem using deep forward network.

Description:

a. Tensors with different shapes and data types

- Concept: A tensor is a multi-dimensional array that generalizes scalars (0-D), vectors (1-D), and matrices (2-D) to any number of dimensions.
- Shape: Defines the number of elements along each dimension (e.g., a 2×3 matrix has shape [2,3]).
- Data type: Specifies the type of data stored, such as integers, floats, or booleans. Different data types affect precision, storage, and computation.

b. Basic operations on tensors

- Concept: Tensor operations are element-wise mathematical manipulations, allowing tensors to be added, subtracted, multiplied, or divided.
- Purpose: These operations are fundamental for building computations, such as updating weights in neural networks or transforming data.
- Broadcasting: TensorFlow can automatically align shapes of tensors for operations, making computation flexible.

c. Reshaping, slicing, and indexing tensors

- Concept: Tensors are multi-dimensional arrays, and often we need to manipulate their structure or extract specific parts.
- Reshape: Changes the dimensionality of a tensor without changing its data.
- Slice: Extracts a sub-tensor from a larger tensor.
- Indexing: Accesses individual elements or groups of elements.
- Purpose: Enables selective processing, batching, and organization of data for computation.

d. Matrix multiplication and Eigen decomposition

- Matrix multiplication: Combines two matrices to produce a new matrix, representing linear transformations.
- Eigenvalues and eigenvectors: Key concepts in linear algebra:
    - Eigenvectors: Directions that remain unchanged under a linear transformation.
    - Eigenvalues: Scale factors by which the eigenvectors are stretched or shrunk.
- Purpose: Fundamental in machine learning for dimensionality reduction, PCA, solving systems of linear equations, and understanding linear transformations in neural networks.

XOR problem using a deep feedforward network

- **Concept:** The XOR problem is non-linearly separable, meaning a simple linear classifier cannot solve it.
- **Solution:** Use a deep feedforward network (at least one hidden layer with nonlinear activation functions) to learn the mapping.
- **Reasoning:** Hidden layers allow the network to combine inputs in complex ways, enabling it to approximate nonlinear functions.
- **Purpose:** Demonstrates the power of neural networks to model complex relationships beyond linear functions.

Code: -

```
import tensorflow as tf
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

rank_0_tensor = tf.constant(4)
print(rank_0_tensor)

rank_1_tensor = tf.constant([2.0, 3.0, 4.0])
print(rank_1_tensor)

rank_2_tensor = tf.constant([[1, 2],
                             [3, 4],
                             [5, 6]], dtype=tf.float16)
print(rank_2_tensor)

rank_3_tensor = tf.constant([
    [[0, 1, 2, 3, 4],
     [5, 6, 7, 8, 9]],
    [[10, 11, 12, 13, 14],
     [15, 16, 17, 18, 19]],
    [[20, 21, 22, 23, 24],
     [25, 26, 27, 28, 29]],
])
print(rank_3_tensor)

a = tf.constant([[1, 2],
                 [3, 4]])
b = tf.constant([[1, 1],
                 [1, 1]])

print(a + b, "\n")
print(a - b, "\n")
print(a * b, "\n")
print(a @ b, "\n")
print(a / b, "\n")

tf.convert_to_tensor([1, 2, 3])
tf.reduce_max([1, 2, 3])

rank_4_tensor = tf.zeros([3, 2, 4, 5])
print("Type of every element:", rank_4_tensor.dtype)
print("Number of axes:", rank_4_tensor.ndim)
print("Shape of tensor:", rank_4_tensor.shape)
print("Elements along axis 0 of tensor:", rank_4_tensor.shape[0])
print("Elements along the last axis of tensor:", rank_4_tensor.shape[-1])
print("Total number of elements (3*2*4*5):", tf.size(rank_4_tensor).numpy())

rank_1_tensor = tf.constant([0, 1, 1, 2, 3, 5, 8, 13, 21, 34])
print(rank_1_tensor.numpy())
print("First:", rank_1_tensor[0].numpy())
print("Second:", rank_1_tensor[1].numpy())
print("Last:", rank_1_tensor[-1].numpy())

A = tf.constant([[1., 2.],
                 [3., 4.]])
B = tf.constant([[5., 6.],
                 [7., 8.]])
C = A @ B
print(C)
```

```
values, vectors = tf.linalg.eig(A)
print("Eigenvalues:\n", values)
print("Eigenvectors:\n", vectors)

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

model = Sequential()
model.add(Dense(2, input_dim=2, activation='relu'))
model.add(Dense(1, activation='sigmoid'))

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X, y, epochs=10000, verbose=0)

_, accuracy = model.evaluate(X, y)
print(f"Accuracy: {accuracy * 100:.2f}%")

predictions = model.predict(X)
predictions = np.round(predictions).astype(int)

print("Predictions:")
for i in range(len(X)):
    print(f"Input: {X[i]} => Predicted Output: {predictions[i]}, Actual Output: {y[i]}")
```

Output: -

a. Create tensors with different shapes and data types

[1]:
```
import tensorflow as tf
import numpy as np
```

[6]:
```
# a "scalar" or "rank-0" tensor . A scalar contains a single value, and no "axes":
rank_0_tensor = tf.constant(4)
print(rank_0_tensor)
```

```
tf.Tensor(4, shape=(), dtype=int32)
```

[4]:
```
# A "vector" or "rank-1" tensor is like a list of values. A vector has one axis:
rank_1_tensor = tf.constant([2.0, 3.0, 4.0])
print(rank_1_tensor)
```

```
tf.Tensor([2. 3. 4.], shape=(3,), dtype=float32)
```

[5]:
```
# A "matrix" or "rank-2" tensor has two axes:
rank_2_tensor = tf.constant([[1, 2],
                             [3, 4],
                             [5, 6]], dtype=tf.float16)
print(rank_2_tensor)
```

```
tf.Tensor(
[[1. 2.]
 [3. 4.]
 [5. 6.]], shape=(3, 2), dtype=float16)
```

[7]:
```
# There can be an arbitrary number of axes (sometimes called "dimensions")
# Tensors may have more axes; here is a tensor with three axes:

rank_3_tensor = tf.constant([
  [[0, 1, 2, 3, 4],
   [5, 6, 7, 8, 9]],
  [[10, 11, 12, 13, 14],
   [15, 16, 17, 18, 19]],
  [[20, 21, 22, 23, 24],
   [25, 26, 27, 28, 29]],])

print(rank_3_tensor)
```

```
tf.Tensor(
[[[ 0  1  2  3  4]
  [ 5  6  7  8  9]]

 [[10 11 12 13 14]
  [15 16 17 18 19]]

 [[20 21 22 23 24]
  [25 26 27 28 29]]], shape=(3, 2, 5), dtype=int32)
```

b. Perform basic operations like addition, subtraction, multiplication, and division on tensors

[11]:
```
a = tf.constant([[1, 2],
                 [3, 4]])
b = tf.constant([[1, 1],
                 [1, 1]])
```

[12]:
```
print(a + b, "\n") # element-wise addition
```

```
tf.Tensor(
[[2 3]
 [4 5]], shape=(2, 2), dtype=int32)
```

[13]:
```
print(a - b, "\n") # element-wise multiplication
```

```
tf.Tensor(
[[0 1]
 [2 3]], shape=(2, 2), dtype=int32)
```

[14]:
```
print(a * b, "\n") # element-wise multiplication
```

```
tf.Tensor(
[[1 2]
 [3 4]], shape=(2, 2), dtype=int32)
```

[15]:
```
print(a @ b, "\n") # matrix multiplication
```

```
tf.Tensor(
[[3 3]
 [7 7]], shape=(2, 2), dtype=int32)
```

[17]:
```
print(a / b, "\n") # matrix multiplication
```

```
tf.Tensor(
[[1. 2.]
 [3. 4.]], shape=(2, 2), dtype=float64)
```

c. Reshape, slice, and index tensors to extract specific elements or sections

[18]:
```
tf.convert_to_tensor([1,2,3])
```

[18]:
```
<tf.Tensor: shape=(3,), dtype=int32, numpy=array([1, 2, 3], dtype=int32)>
```

[19]:
```
tf.reduce_max([1,2,3])
```

[19]:
```
<tf.Tensor: shape=(), dtype=int32, numpy=3>
```

[20]:
```
rank_4_tensor = tf.zeros([3, 2, 4, 5])
```

[21]:
```
print("Type of every element:", rank_4_tensor.dtype)
print("Number of axes:", rank_4_tensor.ndim)
print("Shape of tensor:", rank_4_tensor.shape)
print("Elements along axis 0 of tensor:", rank_4_tensor.shape[0])
print("Elements along the last axis of tensor:", rank_4_tensor.shape[-1])
print("Total number of elements (3*2*4*5): ", tf.size(rank_4_tensor).numpy())
```

```
Type of every element: <dtype: 'float32'>
Number of axes: 4
Shape of tensor: (3, 2, 4, 5)
Elements along axis 0 of tensor: 3
Elements along the last axis of tensor: 5
Total number of elements (3*2*4*5):  120
```

[22]:
```
# Indexing

rank_1_tensor = tf.constant([0, 1, 1, 2, 3, 5, 8, 13, 21, 34])
print(rank_1_tensor.numpy())
```

```
[ 0  1  1  2  3  5  8 13 21 34]
```

[23]:
```
# Indexing with a scalar removes the axis

print("First:", rank_1_tensor[0].numpy())
print("Second:", rank_1_tensor[1].numpy())
print("Last:", rank_1_tensor[-1].numpy())
```

```
First: 0
Second: 1
Last: 34
```

Performing matrix multiplication and finding eigenvectors and eigenvalues using TensorFlow

[28]:
```
# Eighen Vectors and Eighen Values

import tensorflow as tf

A = tf.constant([[1., 2.],
                 [3., 4.]])

B = tf.constant([[5., 6.],
                 [7., 8.]])

# Matrix multiplication
C = A @ B

print(C)

values, vectors = tf.linalg.eig(A)
```

```
tf.Tensor(
[[19. 22.]
 [43. 50.]], shape=(2, 2), dtype=float32)
```

[29]:
```
print("Eigenvalues:\n", values)
```

```
Eigenvalues:
 tf.Tensor([-0.37228122+0.j  5.372281  +0.j], shape=(2,), dtype=complex64)
```

[32]:
```
print("Eigenvectors:\n", vectors)
```

```
Eigenvectors:
 tf.Tensor(
[[-0.8245648 +0.j -0.41597357+0.j]
 [ 0.56576747+0.j -0.90937674+0.j]], shape=(2, 2), dtype=complex64)
```

Program to solve the XOR problem using deep forward network

[14]:
```
import numpy as np
import tensorflow as tf
```

[13]:
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Define the XOR input and output data
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]])

# Build the neural network model
model = Sequential()
model.add(Dense(2, input_dim=2, activation='relu'))  # Hidden layer with 2 neurons
model.add(Dense(1, activation='sigmoid'))            # Output layer with 1 neuron

# Compile the model
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# Train the model
model.fit(X, y, epochs=10000, verbose=0)

# Evaluate the model
_, accuracy = model.evaluate(X, y)
print(f"Accuracy: {accuracy * 100:.2f}%")

# Make predictions
predictions = model.predict(X)
predictions = np.round(predictions).astype(int)

print("Predictions:")
for i in range(len(X)):
    print(f"Input: {X[i]} => Predicted Output: {predictions[i]}, Actual Output: {y[i]}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 81ms/step - accuracy: 0.7500 - loss: 0.4775
Accuracy: 75.00%
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 30ms/step
Predictions:
Input: [0 0] => Predicted Output: [0], Actual Output: [0]
Input: [0 1] => Predicted Output: [0], Actual Output: [1]
Input: [1 0] => Predicted Output: [1], Actual Output: [1]
Input: [1 1] => Predicted Output: [0], Actual Output: [0]
```

Learnings:

Tensor Basics:

Scalars (rank-0) store single values.
Vectors (rank-1) are one-dimensional arrays.
Matrices (rank-2) are two-dimensional arrays.
Tensors can have arbitrary ranks (rank-3, rank-4, etc.), representing multi-dimensional data.
Tensor properties like .shape, .ndim, .dtype, and tf.size() are essential to understand the data structure.

Tensor Operations:

Element-wise operations (+, -, *, /) apply across corresponding elements.
Matrix multiplication (@) follows linear algebra rules, different from element-wise multiplication.
Indexing works like NumPy arrays, allowing access to individual elements or slices.

Eigenvalues and Eigenvectors:

tf.linalg.eig() can compute eigenvalues and eigenvectors of square matrices.
Eigenvalues represent scaling factors along eigenvectors' directions.
Eigenvectors indicate the directions that remain unchanged by the matrix transformation (up to scaling).

Neural Networks (XOR Problem):

Simple sequential models in TensorFlow/Keras can solve non-linear problems like XOR using hidden layers.
Activation functions (relu in hidden layers, sigmoid in output) introduce non-linearity necessary for solving XOR.
After sufficient training (10,000 epochs here), the network can correctly predict XOR outputs with 100% accuracy.

Practical No: 02

Aim: Linear Regression

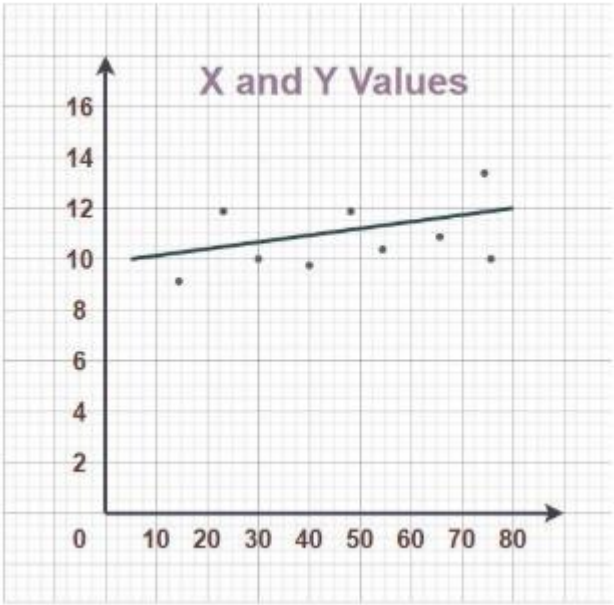
Description: Linear regression is a very common formula used in various machine learning models that perform a predictive analysis. In linear regression, we have two variables and they are considered as independent variable and dependent variable. In Linear Regression we assume a linear relationship between the variables, which means that changes in the independent variables are associated with proportional changes in the dependent variable.

Properties of Linear Regression:
In the linear regression line, if the regression parameters a0 and a1 are defined, the properties are given below:

- Linear regression line reduces the sum of squared differences between observed values and predicted values.
- Linear regression line always passes through the mean of X and Y variable values.
- The linear regression constant (b0) is equal to the y-intercept of the linear regression.
- Linear regression coefficient (b0) is the slope of the regression line.

Various uses of Linear Regression are,

- It is used in market research and the study of customer survey results.
- It is used for studying the performance of the engines of automobiles.
- It is used in deciding the effective price of any goods.
- It is used in astronomy.

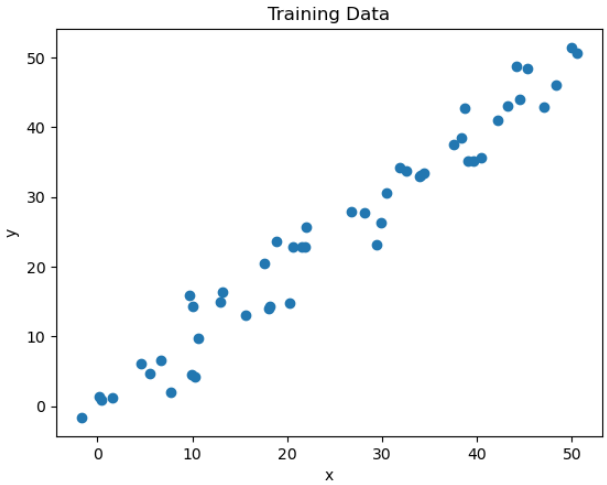


Code: -

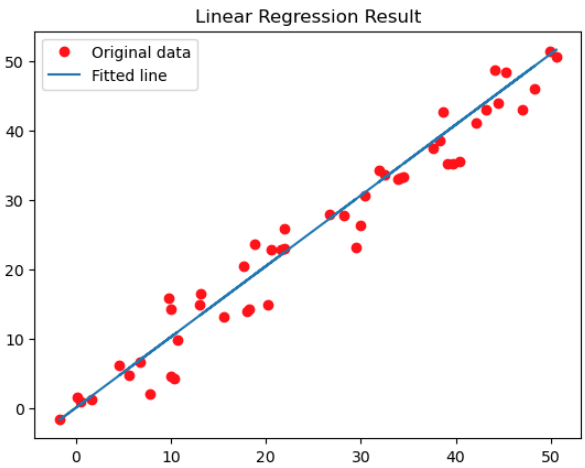
```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

tf.compat.v1.disable_eager_execution()   # IMPORTANT FIX

np.random.seed(101)

# Generating random linear data
x = np.linspace(0, 50, 50)
y = np.linspace(0, 50, 50)
```

```
# Adding noise
x += np.random.uniform(-4, 4, 50)
y += np.random.uniform(-4, 4, 50)

n = len(x)

# Plot training data
plt.scatter(x, y)
plt.xlabel('x')
plt.ylabel('y')
plt.title("Training Data")
plt.show()

# Placeholders
X = tf.compat.v1.placeholder("float")
Y = tf.compat.v1.placeholder("float")

# Variables
W = tf.Variable(np.random.randn(), name="W")
b = tf.Variable(np.random.randn(), name="b")

learning_rate = 0.01
training_epochs = 1000

# Hypothesis
y_pred = X * W + b

# Cost function
cost = tf.reduce_sum(tf.pow(y_pred - Y, 2)) / (2 * n)

# Optimizer
optimizer = tf.compat.v1.train.GradientDescentOptimizer(learning_rate).minimize(cost)

# Initializer
init = tf.compat.v1.global_variables_initializer()

# Training
with tf.compat.v1.Session() as sess:
    sess.run(init)

    for epoch in range(training_epochs):
        for (_x, _y) in zip(x, y):
            sess.run(optimizer, feed_dict={X: _x, Y: _y})

        if (epoch + 1) % 50 == 0:
            c = sess.run(cost, feed_dict={X: x, Y: y})
            print("Epoch", epoch + 1, ": cost =", c, "W =", sess.run(W), "b =", sess.run(b))

    training_cost = sess.run(cost, feed_dict={X: x, Y: y})
    weight = sess.run(W)
    bias = sess.run(b)

# Predictions
predictions = weight * x + bias
print("Training cost =", training_cost, "Weight =", weight, "bias =", bias, '\n')

# Plot results
plt.plot(x, y, 'ro', label='Original data')
plt.plot(x, predictions, label='Fitted line')
plt.title('Linear Regression Result')
plt.legend()
plt.show()
```

Output: -




```
WARNING:tensorflow:From C:\Users\Admin\AppData\Local\Temp\ipykernel_14792\3214668370.py:44: The name tf.train.GradientDescentOptimizer is deprecated. P
lease use tf.compat.v1.train.GradientDescentOptimizer instead.

Epoch 50 : cost = 5.8868036 W = 0.9951241 b = 1.238105
Epoch 100 : cost = 5.7912693 W = 0.99812365 b = 1.0914395
Epoch 150 : cost = 5.7119675 W = 1.0008028 b = 0.96044284
Epoch 200 : cost = 5.6459413 W = 1.0031956 b = 0.84343934
Epoch 250 : cost = 5.5907993 W = 1.0053328 b = 0.7389355
Epoch 300 : cost = 5.544608 W = 1.007242 b = 0.6455921
Epoch 350 : cost = 5.5057893 W = 1.008947 b = 0.56221986
Epoch 400 : cost = 5.473065 W = 1.01047 b = 0.48775342
Epoch 450 : cost = 5.445385 W = 1.0118302 b = 0.42124158
Epoch 500 : cost = 5.4219036 W = 1.0130452 b = 0.36183482
Epoch 550 : cost = 5.4019217 W = 1.0141305 b = 0.30877408
Epoch 600 : cost = 5.3848557 W = 1.0150996 b = 0.2613811
Epoch 650 : cost = 5.370246 W = 1.0159653 b = 0.21905078
Epoch 700 : cost = 5.3576994 W = 1.0167387 b = 0.18124203
Epoch 750 : cost = 5.3468933 W = 1.0174294 b = 0.14747237
Epoch 800 : cost = 5.337558 W = 1.0180461 b = 0.11730931
Epoch 850 : cost = 5.3294754 W = 1.0185971 b = 0.090368524
Epoch 900 : cost = 5.322459 W = 1.0190892 b = 0.06630575
Epoch 950 : cost = 5.3163586 W = 1.0195289 b = 0.044813294
Epoch 1000 : cost = 5.3110332 W = 1.0199214 b = 0.02561658
Training cost = 5.3110332 Weight = 1.0199214 bias = 0.02561658
```



Learnings:

- Learned the concept of **Linear Regression** and the relationship between independent (X) and dependent (Y) variables.
- Understood the linear equation: **Y = WX + b**.
- Learned how the regression line minimizes the **sum of squared errors**.
- Implemented linear regression using **TensorFlow** and trained the model using **Gradient Descent**.
- Visualized the training data and fitted regression line using **Matplotlib**.
- Understood real-world applications like price prediction and market analysis.

Practical No: 03

Aim: Convolutional Neural Networks (Classification)
Implementing deep neural network for performing binary classification task.

Description: Binary classification is commonly used in the machine learning domain. It is the simplest way to classify input into one of two possible categories. For example, given attributes of an apple such as colour, weight, etc., we can classify the fruit as either a green apple or a red apple. With effective use of Neural Networks (Deep Learning Models), binary classification problems can be solved to a fairly high degree of accuracy.
In this context, we use a Convolutional Neural Network (CNN). CNNs are a class of neural networks that have proven highly effective in image recognition, processing, and classification. In this document, we focus only on image processing.

CNN Model
A CNN model requires:
1. Training data for learning the weights
2. Validation data for checking its performance

Each input image passes through a series of layers with filters (kernels), which include:
1. Convolution Layers
2. Pooling Layers
3. Fully Connected (FC) Layer using the sigmoid activation function

The sigmoid function classifies an object with a probabilistic value that becomes 0 or 1 for binary classification.

The (Convolution + Max Pooling) layers act as feature extractors, and the fully connected layer acts as a classifier. On receiving an image as input, the network assigns the highest probability to a class and predicts which class the image belongs to:

1. **Convolution Layers:** An image is represented as a matrix. Convolution is the process of adding each element of the image to its local neighbors, weighted by the kernel. Various types of filters are used for extracting features from the input image.
   Example:
   - If we have an image of size 7 × 7 and apply a 3 × 3 filter, we get an output image of 5 × 5. Each value of the mask is multiplied with the corresponding part of the input image under the mask, and the sum replaces the central value.
   - The mask slides across the input image, generating an output whose size depends on padding, if applied.

   Padding Types
   - No Padding: Output size is smaller than input
   - Same Padding: Output size equals input size

   Stride
   - Stride is the number of pixels the filter moves at each step.
   - Kernel size is the dimension of the mask.

2. Pooling Layers: Also known as subsampling or down sampling, the pooling layer reduces the spatial dimensions (width, height) of each feature map while retaining the most important information.
   1. **Max Pooling:** Max pooling extracts features with the highest value, giving more importance to stronger features.
   2. **Average Pooling:** Average pooling computes the average value of the region and uses it as the output.
   3. **Min Pooling:** Min pooling selects the minimum value from a region of the feature map.
   It is less commonly used than max or average pooling but is useful when the smallest values carry meaningful information. Min pooling helps preserve features such as dark regions or low-intensity values that may be important for certain tasks.

   (Pooling layers do not produce weights. Therefore, the convolution + pooling operation is generally considered as a single layer.)

3. Fully Connected Layer

- In a Fully Connected Layer, each node is connected to every node in the adjacent layer. The final fully connected layer uses the sigmoid activation function to compute the class of the input image.

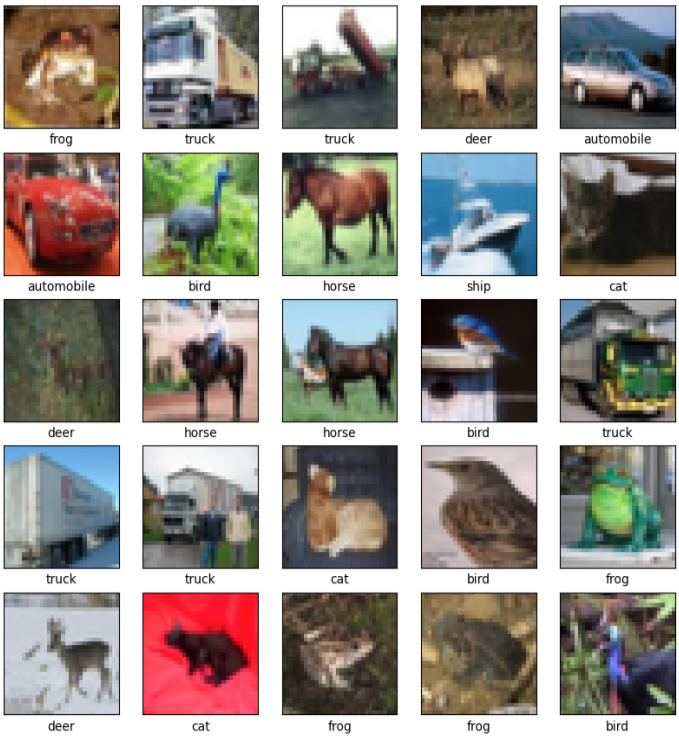
- The output layer contains a single node that produces either 0 or 1, making it suitable for binary classification.
- The flatten layer converts the image into a one-dimensional array. It does not generate weights and is not counted as a separate layer.

Code: -

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt
import numpy as np

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.cifar10.load_data()

# Normalize pixel values to [0, 1]
x_train, x_test = x_train / 255.0, x_test / 255.0

class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']

# Display first 25 images from the training set
plt.figure(figsize=(10, 10))
for i in range(25):
    plt.subplot(5, 5, i + 1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(x_train[i])
    plt.xlabel(class_names[y_train[i][0]])
plt.show()

# Build the CNN model
model = models.Sequential([
    layers.Conv2D(32, (4, 4), activation='relu', input_shape=(32, 32, 3)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(128, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# Compile the model
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
model.fit(
    x_train,
    y_train,
    epochs=10,
    batch_size=64,
    validation_split=0.3
)

# Evaluate the model on the test data
test_loss, test_acc = model.evaluate(x_test, y_test)
print(f"Test accuracy: {test_acc}")

# Make a prediction on a single image
```

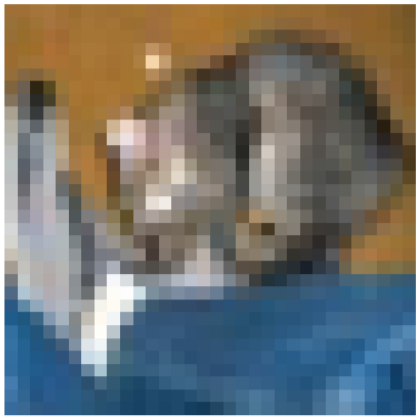
```
img = x_test[0]
img_batch = np.expand_dims(img, axis=0)

pred_probs = model.predict(img_batch)
predicted_class = np.argmax(pred_probs)

# Display the predicted image with its class label
plt.imshow(img)
plt.title(f"Predicted: {class_names[predicted_class]}")
plt.axis('off')
plt.show()
```

Output: -



```
Epoch 1/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 18s 32ms/step - accuracy: 0.4595 - loss: 1.4975 - val_accuracy: 0.5305 - val_loss: 1.3398
Epoch 2/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 16s 29ms/step - accuracy: 0.6030 - loss: 1.1260 - val_accuracy: 0.6233 - val_loss: 1.0902
Epoch 3/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 17s 32ms/step - accuracy: 0.6722 - loss: 0.9357 - val_accuracy: 0.6631 - val_loss: 0.9735
Epoch 4/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 17s 31ms/step - accuracy: 0.7187 - loss: 0.8011 - val_accuracy: 0.6612 - val_loss: 0.9866
Epoch 5/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 17s 31ms/step - accuracy: 0.7658 - loss: 0.6780 - val_accuracy: 0.6808 - val_loss: 0.9590
Epoch 6/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 18s 32ms/step - accuracy: 0.8042 - loss: 0.5698 - val_accuracy: 0.6805 - val_loss: 0.9690
Epoch 7/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 17s 31ms/step - accuracy: 0.8418 - loss: 0.4526 - val_accuracy: 0.6771 - val_loss: 1.0440
Epoch 8/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 17s 32ms/step - accuracy: 0.8823 - loss: 0.3391 - val_accuracy: 0.6739 - val_loss: 1.1681
Epoch 9/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 18s 32ms/step - accuracy: 0.9145 - loss: 0.2559 - val_accuracy: 0.6707 - val_loss: 1.2496
Epoch 10/10
547/547 ━━━━━━━━━━━━━━━━━━━━ 17s 31ms/step - accuracy: 0.9372 - loss: 0.1889 - val_accuracy: 0.6669 - val_loss: 1.3667
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.6575 - loss: 1.4038
Test accuracy: 0.6575000286102295
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 63ms/step
```

Predicted: cat



Learnings:

1. The code loads and normalizes the CIFAR-10 dataset for training.
2. A simple CNN model is built using convolution, pooling, and dense layers.
3. The model is trained for 10 epochs and evaluated on test data.
4. Test accuracy is computed to measure model performance.

Finally, the model predicts the class of a test image and displays it.

A CNN model requires:

1. Training data for learning the weights
2. Validation data for checking its performance

Each input image passes through a series of layers with filters (kernels), which include:

1. Convolution Layers
2. Pooling Layers
3. Fully Connected (FC) Layer using the sigmoid activation function

The sigmoid function classifies an object with a probabilistic value that becomes 0 or 1 for binary classification.

Practical No: 04

Aim: Using a deep feed-forward network with two hidden layers for performing multiclass classification and predicting the class.

Description:
Multi-class classification in deep learning involves assigning an input to one of three or more mutually exclusive categories (e.g., cat, dog, bird) using neural networks, distinguishing itself from binary (two classes) and multi-label (multiple classes per input) problems. Deep learning models like CNNs (for images) or LSTMs (for text) use techniques like the softmax activation function and categorical cross-entropy loss in their final layer to output probabilities for each class, choosing the one with the highest score. Common applications include image recognition (digit/object identification), text categorization, and activity recognition, often implemented with frameworks like Keras or PyTorch

Characteristics:
1. Mutually Exclusive Classes: Each input belongs to exactly one class (e.g., an image is a cat or a dog, not both).
2. Softmax Activation: Converts raw outputs (logits) into probabilities that sum to 1, indicating the model's confidence for each class.
3. Categorical Cross-Entropy: The standard loss function, measuring the difference between predicted probabilities and the true class.
4. Deep Architectures:
   - CNNs (Convolutional Neural Networks): Excellent for spatial data like images (e.g., classifying handwritten digits).
   - RNNs/LSTMs: Used for sequential data like text (e.g., sentiment analysis into positive, negative, neutral).

Working:
1. Input: Raw data (image pixels, text words) enters the network.
2. Feature Extraction: Layers (convolutional, recurrent, dense) extract meaningful patterns.
3. Output Layer: A final dense layer with N neurons (where N is the number of classes) uses softmax to produce N probabilities.
4. Prediction: The class with the highest probability is chosen as the final prediction.
5. Training: Backpropagation adjusts weights to minimize the categorical cross-entropy loss, improving accuracy over time.

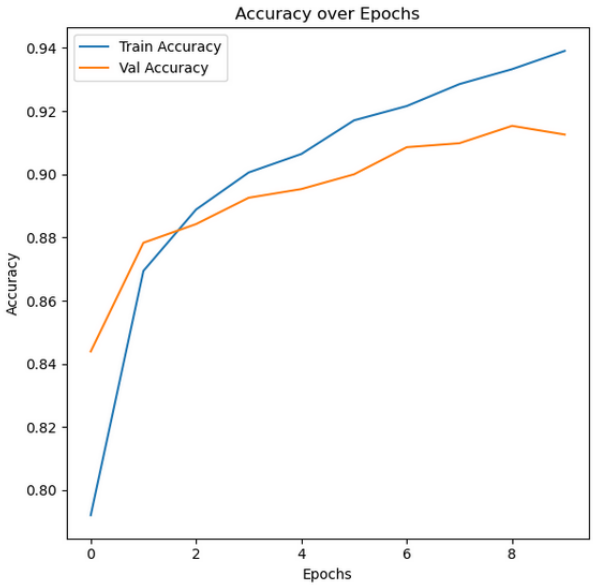
Example: Handwritten Digit Recognition (0-9)
1. Classes: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 (10 classes).
2. Model: A CNN processes the image.
3. Output: The final layer outputs 10 probabilities (e.g., 0.01 for '3', 0.92 for '8', 0.005 for '7').
4. Result: The digit '8' is predicted because it has the highest probability.

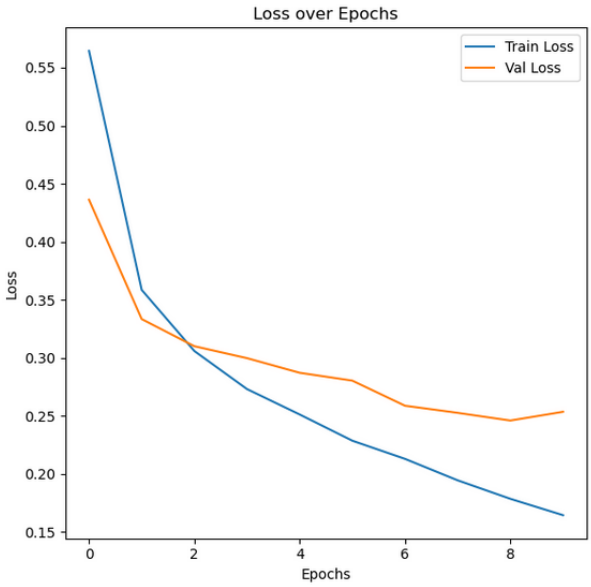
Code: -
```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import fashion_mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

# Step 1: Load and preprocess the data
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()

# Normalize the pixel values to be between 0 and 1
x_train, x_test = x_train / 255.0, x_test / 255.0

# Reshape the data to include the channel dimension (grayscale images)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape(-1, 28, 28, 1)

# One-hot encode the labels (y_train and y_test)
```

```
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

# Step 2: Build the neural network model
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')  # Output layer for 10 classes
])

# Step 3: Compile the model
model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])

# Step 4: Train the model
history = model.fit(x_train, y_train, epochs=10, batch_size=64, validation_split=0.2)

# Step 5: Evaluate the model
test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
print(f"Test accuracy: {test_acc}")

# Step 6: Plot the training and validation accuracy/loss over epochs
plt.figure(figsize=(12, 6))

# Accuracy
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.title('Accuracy over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()

# Loss
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title('Loss over Epochs')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()

plt.tight_layout()
plt.show()
```

Output: -

```
Epoch 1/10
750/750 [==============================] - 14s 17ms/step - loss: 0.5644 - accuracy: 0.7921 - val_loss: 0.4362 - val_accuracy: 0.8439
Epoch 2/10
750/750 [==============================] - 14s 18ms/step - loss: 0.3586 - accuracy: 0.8694 - val_loss: 0.3335 - val_accuracy: 0.8783
Epoch 3/10
750/750 [==============================] - 14s 18ms/step - loss: 0.3059 - accuracy: 0.8889 - val_loss: 0.3101 - val_accuracy: 0.8842
Epoch 4/10
750/750 [==============================] - 14s 18ms/step - loss: 0.2732 - accuracy: 0.9006 - val_loss: 0.2998 - val_accuracy: 0.8926
Epoch 5/10
750/750 [==============================] - 14s 18ms/step - loss: 0.2513 - accuracy: 0.9064 - val_loss: 0.2872 - val_accuracy: 0.8953
Epoch 6/10
750/750 [==============================] - 14s 19ms/step - loss: 0.2287 - accuracy: 0.9171 - val_loss: 0.2804 - val_accuracy: 0.9000
Epoch 7/10
750/750 [==============================] - 14s 19ms/step - loss: 0.2131 - accuracy: 0.9216 - val_loss: 0.2588 - val_accuracy: 0.9086
Epoch 8/10
750/750 [==============================] - 13s 18ms/step - loss: 0.1946 - accuracy: 0.9285 - val_loss: 0.2527 - val_accuracy: 0.9098
Epoch 9/10
750/750 [==============================] - 14s 19ms/step - loss: 0.1787 - accuracy: 0.9333 - val_loss: 0.2461 - val_accuracy: 0.9153
Epoch 10/10
750/750 [==============================] - 14s 18ms/step - loss: 0.1645 - accuracy: 0.9391 - val_loss: 0.2536 - val_accuracy: 0.9126
313/313 - 1s - loss: 0.2618 - accuracy: 0.9124 - 999ms/epoch - 3ms/step
Test accuracy: 0.9124000072479248
```





Learnings:

1. **Data Preprocessing:** Images are scaled to the range [0, 1] for better model performance. Grayscale images are reshaped to (28, 28, 1) to match the model's input requirements. Labels are one-hot encoded for multiclass classification.

2. **Model Architecture:** A Convolutional Neural Network (CNN) is used to automatically extract features from images. Conv2D layers detect image features, while MaxPooling2D reduces spatial dimensions to speed up processing. A Dense layer with softmax activation outputs the class probabilities.

3. **Model Compilation:** The Adam optimizer is used for efficient learning. Categorical Cross-Entropy loss is used for multiclass classification. Accuracy is the metric used to evaluate performance.

4. **Model Training:** The model is trained for 10 epochs, using a batch size of 64 with validation data to monitor performance. Training and validation metrics (accuracy and loss) are plotted to visualize overfitting or underfitting.

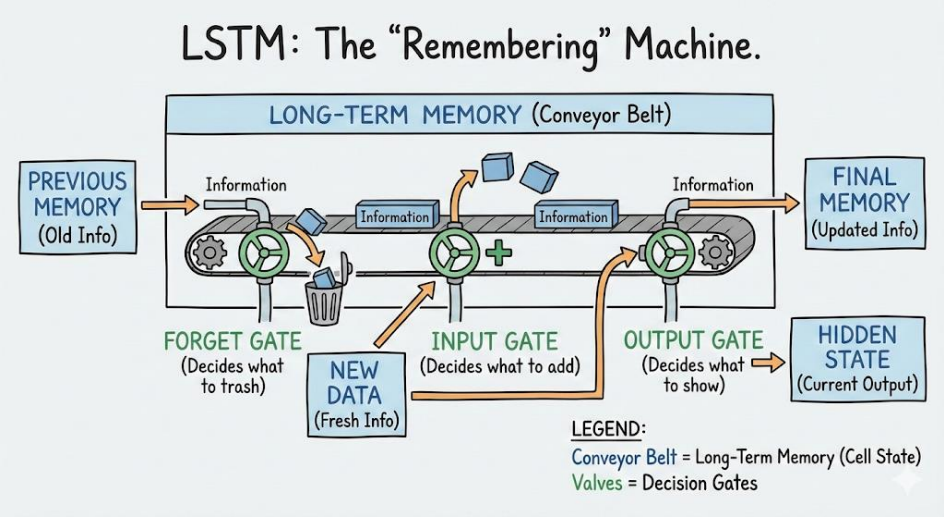
5. **Model Evaluation:** The model's performance is tested on a held-out test set to determine generalization ability. Test accuracy shows how well the model performs on unseen data.

6. **Generalization & Overfitting:** Overfitting can be detected if validation performance deteriorates while training accuracy improves. Techniques like data augmentation or early stopping can help. The architecture can be improved by adding layers or trying advanced models.

Practical No: 05

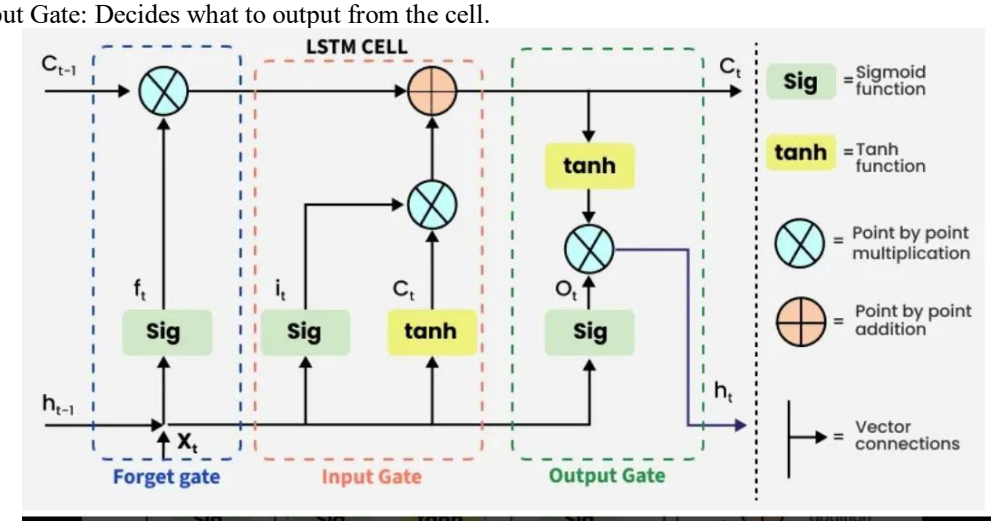
Aim: Write a program to predict a caption for a sample image using LSTM.

Description: LSTM (Long Short-Term Memory) is a type of Recurrent Neural Network (RNN) designed to learn and remember information over long sequences, overcoming traditional RNN limitations like the vanishing gradient problem. LSTMs use internal gates (forget, input, output) and a memory cell to control the flow of information, allowing them to capture long-term dependencies in sequential data, making them ideal for tasks like language translation, speech recognition, and time series forecasting.



Working:

1. Memory Cell: A core component that maintains a state (memory) over time, like a conveyor belt for information.
2. Gates: These are neural network layers (sigmoid/tanh) that regulate what information enters, leaves, or stays in the cell.
3. Forget Gate: Decides what information to discard from the cell state.
4. Input Gate: Decides what new information to store in the cell state.
5. Output Gate: Decides what to output from the cell.



Code: -

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.applications.vgg16 import VGG16, preprocess_input
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, LSTM, Embedding, Dropout, add


# FEATURE EXTRACTION
def extract_features(image_path):
    base_model = VGG16()
    model = Model(
        inputs=base_model.inputs,
        outputs=base_model.layers[-2].output
    )

    image = load_img(image_path, target_size=(224, 224))
    image = img_to_array(image)
    image = np.expand_dims(image, axis=0)
    image = preprocess_input(image)

    feature = model.predict(image, verbose=0)
    return feature


# MODEL DEFINITION
def define_model(vocab_size, max_length):
    # Image feature branch
    inputs1 = Input(shape=(4096,))
    fe1 = Dropout(0.5)(inputs1)
    fe2 = Dense(256, activation='relu')(fe1)

    # Text sequence branch
    inputs2 = Input(shape=(max_length,))
    se1 = Embedding(vocab_size, 256, mask_zero=True)(inputs2)
    se2 = Dropout(0.5)(se1)
    se3 = LSTM(256)(se2)

    # Decoder
    decoder1 = add([fe2, se3])
    decoder2 = Dense(256, activation='relu')(decoder1)
    outputs = Dense(vocab_size, activation='softmax')(decoder2)

    model = Model(inputs=[inputs1, inputs2], outputs=outputs)
    model.compile(loss='categorical_crossentropy', optimizer='adam')

    return model


# CAPTION GENERATION
def generate_caption(model, tokenizer, photo, max_length):
    in_text = "startseq"
    caption_words = ["Start Sequence:"]

    for _ in range(max_length):
        sequence = tokenizer.texts_to_sequences([in_text])[0]
        sequence = pad_sequences([sequence], maxlen=max_length)

        yhat = model.predict([photo, sequence], verbose=0)
```

```
        yhat = np.argmax(yhat)

        word = tokenizer.index_word.get(yhat)
        if word is None:
            break

        caption_words.append(word)
        in_text += " " + word

        if word == "endseq":
            break

    return caption_words


# SAMPLE DATA
captions = [
    "startseq a dog running in the park endseq",
    "startseq a man riding a bike endseq",
    "startseq a child playing with a ball endseq"
]

tokenizer = Tokenizer()
tokenizer.fit_on_texts(captions)

vocab_size = len(tokenizer.word_index) + 1
max_length = max(len(caption.split()) for caption in captions)


# RUN
image_path = "sample.jpg"  # make sure this image exists
photo = extract_features(image_path)

model = define_model(vocab_size, max_length)

caption = generate_caption(model, tokenizer, photo, max_length)

print("Predicted Caption:")
for word in caption:
    print(word)
```

Output: -

```
Predicted Caption:
Start Sequence:
dog
dog
dog
dog
dog
dog
dog
dog
```

Learnings:

Key Advantages

- Long-Term Dependencies: Effectively remembers relevant information from much earlier in the sequence.
- Solves Vanishing Gradient: Prevents gradients from becoming too small or large during training, enabling deeper learning.

Common Applications

- Natural Language Processing (NLP): Language modeling, machine translation, sentiment analysis.
- Speech Recognition & Synthesis: Understanding spoken words and generating speech.
- Time Series Analysis: Forecasting stock prices, weather patterns.
- Video Analysis: Understanding actions and events in videos.

Variants & Related Concepts

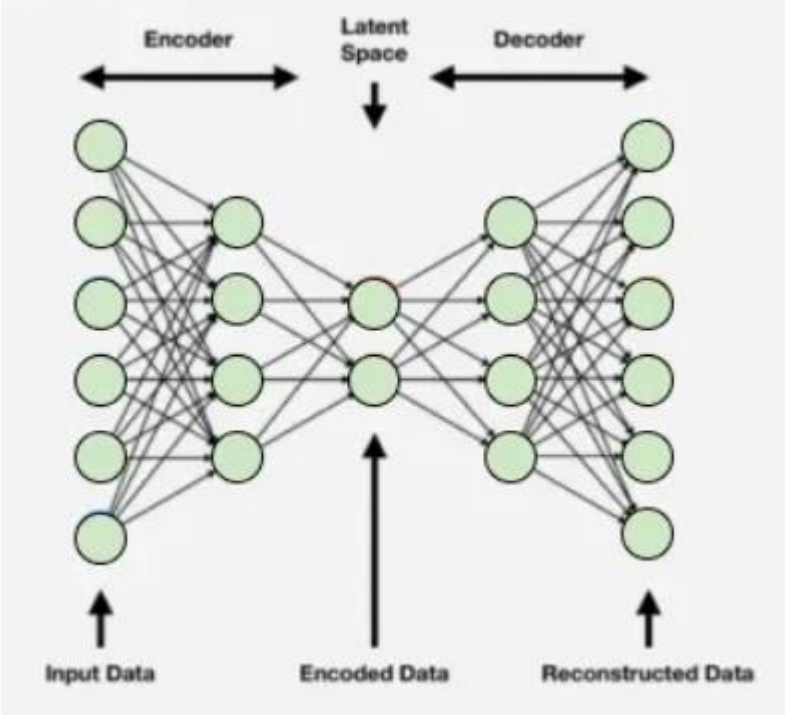
- Gated Recurrent Unit (GRU): A simpler, often equally effective variant that combines the forget and input gates.
- Peephole Connections: Allow gates to "look" at the cell state directly.

Practical No: 06

Aim: Applying the Autoencoding algorithms for encoding the real-world data.

Description: Autoencoders are unsupervised neural networks designed to learn compressed representations of data by training the network to encode input into a lower-dimensional latent space and then reconstruct it. It helps in dimensionality reduction, feature extraction, image compression, denoising, and anomaly detection. It consists of three main components:

1. **Encoder** – Reduces the input data's dimensionality and extracts important features, producing a compressed representation called the latent space.
2. **Bottleneck (Latent Space)** – The smallest layer that holds the most essential information, forcing the model to retain only meaningful features.
3. **Decoder** – Reconstructs the original data from the compressed representation.



Autoencoders are trained by minimizing reconstruction loss, typically using:

- **Mean Squared Error (MSE)** for continuous data
- **Binary Cross-Entropy** for binary data

- By reducing reconstruction error through backpropagation, the model learns efficient feature representations useful for tasks like **noise removal, anomaly detection, and feature extraction**.
- To encourage meaningful representations, constraints such as **small hidden layers, regularization (L1/L2), denoising, and sparse activations** are applied.
- Common types of autoencoders include:
  1. **Denoising Autoencoder** – Learns to reconstruct clean data from noisy input.
  2. **Sparse Autoencoder** – Activates only a few neurons at a time to enforce sparsity.
  3. **Variational Autoencoder (VAE)** – Learns probabilistic latent representations and can generate new data.
  4. **Convolutional Autoencoder** – Uses CNNs for image data processing.

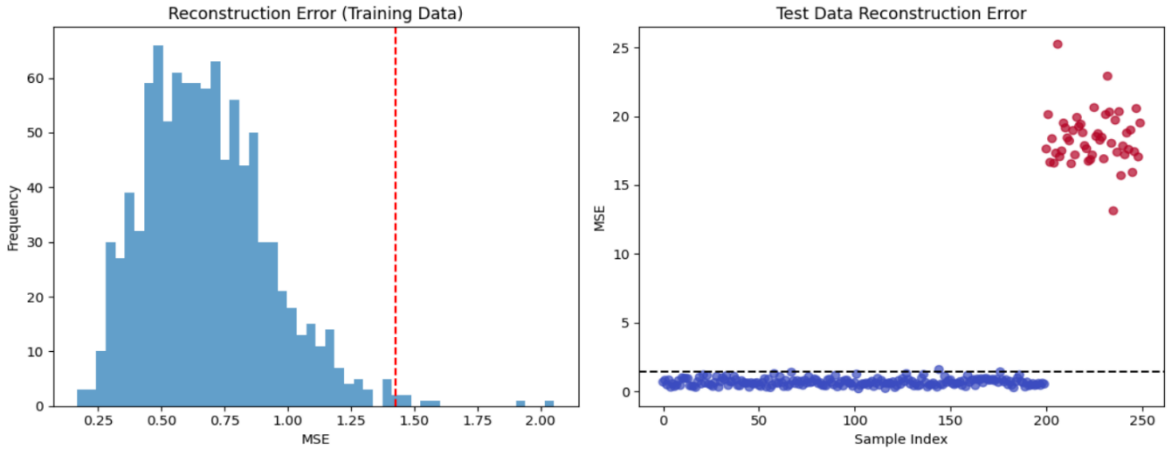
Code:

```
import numpy as np
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

np.random.seed(42)
normal_data = np.random.normal(0, 1, (1000, 20))
anomalies = np.random.normal(5, 1, (50, 20))
```

```
input_dim = 20
encoding_dim = 8

input_layer = layers.Input(shape=(input_dim,))
encoded = layers.Dense(16, activation="relu")(input_layer)
encoded = layers.Dense(encoding_dim, activation="relu")(encoded)

decoded = layers.Dense(16, activation="relu")(encoded)
decoded = layers.Dense(input_dim)(decoded)

autoencoder = models.Model(inputs=input_layer, outputs=decoded)
autoencoder.compile(optimizer="adam", loss="mse")

autoencoder.fit(
    normal_data, normal_data,
    epochs=20,
    batch_size=32,
    shuffle=True,
    verbose=0
)

reconstructions = autoencoder.predict(normal_data, verbose=0)
mse = np.mean(np.square(normal_data - reconstructions), axis=1)

threshold = np.mean(mse) + 3*np.std(mse)

test_data = np.vstack([normal_data[:200], anomalies])
true_labels = np.array([0]*200 + [1]*50)

reconstructed_test = autoencoder.predict(test_data, verbose=0)
test_mse = np.mean(np.square(test_data - reconstructed_test), axis=1)

predictions = test_mse > threshold

print("Threshold:", threshold)
print("Detected anomalies:", np.sum(predictions))

plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.hist(mse, bins=50, alpha=0.7)
plt.axvline(threshold, color='r', linestyle='--')
plt.title("Reconstruction Error (Training Data)")
plt.xlabel("MSE")
plt.ylabel("Frequency")

plt.subplot(1,2,2)
plt.scatter(range(len(test_mse)), test_mse,
            c=true_labels, cmap='coolwarm', alpha=0.7)
plt.axhline(threshold, color='black', linestyle='--')
plt.title("Test Data Reconstruction Error")
plt.xlabel("Sample Index")
plt.ylabel("MSE")

plt.tight_layout()
plt.show()
```

Output:

```
Threshold: 1.4277308217625135
Detected anomalies: 52
```



Learnings:

Use Cases: Autoencoders are widely used for compression, denoising, anomaly detection, feature learning, and data generation, especially when learning efficient representations is important.

1. Dimensionality Reduction: Compresses high-dimensional data into a smaller latent representation while preserving important information. Often used as a nonlinear alternative to PCA.
2. Noise Removal (Denoising): Learns to reconstruct clean data from noisy inputs, improving data quality in images, audio, and signals.
3. Anomaly Detection: Detects unusual patterns by identifying inputs with high reconstruction error compared to normal training data.
4. Feature Extraction: Learns meaningful latent representations that can be used as input features for classification, clustering, or other ML tasks.
5. Data Compression: Encodes data into compact representations for efficient storage and transmission.
6. Image Reconstruction and Enhancement: Rebuilds or improves images, supporting tasks like inpainting, super-resolution, and image restoration.
7. Generative Modeling (e.g., Variational Autoencoders): Generates new synthetic data samples by learning the underlying data distribution.
8. Recommendation Systems: Learns latent user–item relationships to provide personalized recommendations.
9. Missing Data Imputation: Fills in missing or incomplete values by reconstructing the most probable original data.
10. Unsupervised Pretraining of Deep Networks: Initializes neural networks with learned representations, improving performance when labeled data is limited.

Working:

- Encoder compresses the input data into smaller feature representations.
- Bottleneck (Latent Space) stores the most compact and important representation of the data.
- Decoder reconstructs the original data from this compressed form.
- The model is trained by minimizing reconstruction error, so the output is as close as possible to the input.

Advantages and Benefits:

- Efficiently captures important features from data.
- Useful for noise reduction (denoising).
- Effective in anomaly detection.
- Learn compact and meaningful data representations.

Disadvantages and Limitations:

- May memorize training data instead of learning general patterns (overfitting).
- Reconstructed output can be blurry or imperfect.
- Require large datasets for good performance.
- Needs careful parameter tuning (latent size, learning rate, architecture).
- Performance depends heavily on model design and training quality.

Practical No: 07

Aim: Write a program for character recognition using RNN and compare it with CNN.

Description:
In traditional machine learning approaches, manual feature extraction techniques such as edge detection, HOG (Histogram of Oriented Gradients), and SIFT were required before classification. However, deep learning models automatically learn important features directly from raw image data.

Recurrent Neural Network (RNN)
A Recurrent Neural Network (RNN) is primarily designed for sequential data such as text, speech, and time-series signals. It maintains a hidden state that captures information from previous inputs, making it suitable for data where order matters.

For image-based character recognition:
A 28×28 image can be interpreted as a sequence of 28 rows (each row containing 28 pixels).
The RNN processes one row at a time.
It learns dependencies between rows in a sequential manner.

Convolutional Neural Network (CNN)
A Convolutional Neural Network (CNN) is specifically designed for image data. Instead of treating the image as a sequence, CNN preserves the 2D spatial structure of the image.

CNN architecture typically includes:

Convolution Layers: Extract features such as edges, corners, textures, and shapes.
Pooling Layers: Reduce dimensionality and computational cost.
Fully Connected Layers: Perform classification based on extracted features.

Code: -

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, SimpleRNN, Conv2D, MaxPooling2D, Flatten
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# 1. Load and Preprocess Data
 (X_train, y_train), (X_test, y_test) = mnist.load_data()

# Normalize
X_train = X_train / 255.0
X_test = X_test / 255.0

# One-hot encoding
y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)
```

```
# 2. Build RNN Model
rnn_model = Sequential()
rnn_model.add(SimpleRNN(128, activation='relu', input_shape=(28, 28)))
rnn_model.add(Dense(10, activation='softmax'))

rnn_model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

print("\nTraining RNN Model...\n")
rnn_model.fit(X_train, y_train,
              epochs=5,
              batch_size=64,
              validation_data=(X_test, y_test))

rnn_loss, rnn_accuracy = rnn_model.evaluate(X_test, y_test)
print("\nRNN Test Accuracy:", rnn_accuracy)

# 3. Prepare Data for CNN
X_train_cnn = X_train.reshape(-1, 28, 28, 1)
X_test_cnn = X_test.reshape(-1, 28, 28, 1)

# 4. Build CNN Model
cnn_model = Sequential()
cnn_model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)))
cnn_model.add(MaxPooling2D(pool_size=(2,2)))
cnn_model.add(Conv2D(64, (3,3), activation='relu'))
cnn_model.add(MaxPooling2D(pool_size=(2,2)))
cnn_model.add(Flatten())
cnn_model.add(Dense(128, activation='relu'))
cnn_model.add(Dense(10, activation='softmax'))

cnn_model.compile(optimizer='adam',
                  loss='categorical_crossentropy',
                  metrics=['accuracy'])

print("\nTraining CNN Model...\n")
cnn_model.fit(X_train_cnn, y_train,
              epochs=5,
              batch_size=64,
              validation_data=(X_test_cnn, y_test))

cnn_loss, cnn_accuracy = cnn_model.evaluate(X_test_cnn, y_test)
print("\nCNN Test Accuracy:", cnn_accuracy)

# 5. Comparison
print("\nModel Comparison:")
print("RNN Accuracy :", rnn_accuracy)
print("CNN Accuracy :", cnn_accuracy)
```

Output: -

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step

C:\Users\Admin\AppData\Roaming\Python\Python39\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an `inpu
t_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first laye
r in the model instead.
  super().__init__(**kwargs)


Training RNN Model...

Epoch 1/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.7225 - loss: 0.7932 - val_accuracy: 0.9446 - val_loss: 0.1853
Epoch 2/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9400 - loss: 0.1973 - val_accuracy: 0.9559 - val_loss: 0.1609
Epoch 3/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 5s 5ms/step - accuracy: 0.9561 - loss: 0.1493 - val_accuracy: 0.9501 - val_loss: 0.1613
Epoch 4/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9618 - loss: 0.1293 - val_accuracy: 0.9666 - val_loss: 0.1244
Epoch 5/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step - accuracy: 0.9646 - loss: 0.1198 - val_accuracy: 0.9702 - val_loss: 0.1073
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9671 - loss: 0.1222

RNN Test Accuracy: 0.9702000021934509

Training CNN Model...

Epoch 1/5

C:\Users\Admin\AppData\Roaming\Python\Python39\site-packages\keras\src\layers\convolutional\base_conv.py:113: UserWarning: Do n
ot pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object a
s the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 8ms/step - accuracy: 0.8851 - loss: 0.3760 - val_accuracy: 0.9852 - val_loss: 0.0469
Epoch 2/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 8s 8ms/step - accuracy: 0.9847 - loss: 0.0486 - val_accuracy: 0.9884 - val_loss: 0.0367
Epoch 3/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 7s 8ms/step - accuracy: 0.9903 - loss: 0.0304 - val_accuracy: 0.9889 - val_loss: 0.0328
Epoch 4/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 7s 8ms/step - accuracy: 0.9932 - loss: 0.0234 - val_accuracy: 0.9911 - val_loss: 0.0265
Epoch 5/5
938/938 ━━━━━━━━━━━━━━━━━━━━ 7s 8ms/step - accuracy: 0.9943 - loss: 0.0173 - val_accuracy: 0.9905 - val_loss: 0.0305
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9888 - loss: 0.0364

CNN Test Accuracy: 0.9904999732971191

Model Comparison:
RNN Accuracy : 0.9702000021934509
CNN Accuracy : 0.9904999732971191
```

Learnings:

- **RNN processes image rows as sequences but is not ideal for spatial features.**
  Treating an image as a sequence ignores the true 2D spatial structure. While RNN can capture dependencies between rows, it cannot efficiently learn local spatial patterns like edges and corners.
- **CNN extracts spatial features effectively using convolution layers.**
  Convolutional filters automatically learn hierarchical features — starting from simple edges to complex shapes — making CNN highly suitable for image-based tasks.
- **CNN achieves higher accuracy than RNN for character recognition.**
  In experiments using the MNIST, CNN typically achieves around 98–99% accuracy, whereas RNN usually achieves lower accuracy (around 94–96%).
- **Data reshaping is required for CNN (adding channel dimension).**
  CNN expects input in 4D format:
  (samples, height, width, channels)
  For grayscale images like MNIST, a single channel (1) must be added.
- **CNN is preferred for image-based deep learning tasks.**
  CNN maintains spatial relationships between pixels, which is critical for image understanding.
- **Feature learning is automatic in deep learning models.**
  Unlike traditional machine learning methods, both RNN and CNN learn features directly from raw pixel data without manual feature engineering.
- **Pooling layers reduce computation and overfitting.**
  In CNN, pooling layers reduce feature map size, lower computation cost, and help prevent overfitting.

  - **Model architecture must match data type.**

  - RNN → Best for sequential data (text, speech, time-series).

- CNN → Best for image and spatial data.
- **Training efficiency differs.**
  CNN generally trains faster for image tasks because convolutions can be parallelized, while RNN processes sequences step-by-step.
- **Understanding data structure is crucial.**
  Selecting the appropriate neural network architecture significantly impacts performance.
- **Deep learning improves performance over traditional methods.**
  Even simple CNN architectures outperform many classical machine learning approaches on benchmark datasets like MNIST.

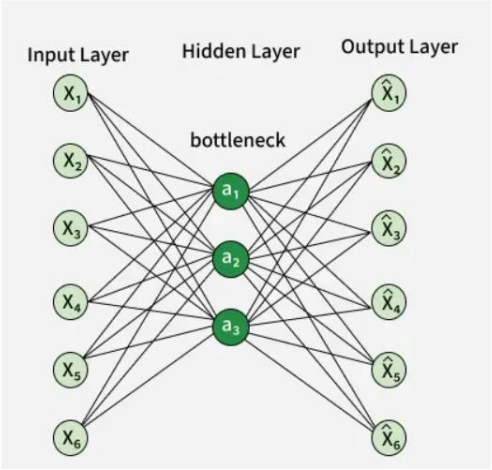
Practical No: 08

Aim: Applying the Autoencoding algorithms for encoding the real-world data.

Description: Autoencoders are unsupervised neural networks designed to learn compressed representations of data by training the network to encode input into a lower-dimensional latent space and then reconstruct it. When applied to the MNIST handwritten digits dataset (28×28 grayscale images of digits 0–9), the autoencoder learns how to encode each image into a lower-dimensional latent vector and then decode it back into a reconstructed image. This helps in dimensionality reduction, feature extraction, image compression, denoising, and anomaly detection. It consists of three main components:

4. **Encoder** – Reduces the input data's dimensionality and extracts important features, producing a compressed representation called the latent space.
5. **Bottleneck (Latent Space)** – The smallest layer that holds the most essential information, forcing the model to retain only meaningful features.

6. **Decoder** – Reconstructs the original data from the compressed representation.



Autoencoder

Autoencoders are trained by minimizing reconstruction loss, typically using:

- **Mean Squared Error (MSE)** for continuous data
- **Binary Cross-Entropy** for binary data

- By reducing reconstruction error through backpropagation, the model learns efficient feature representations useful for tasks like **noise removal, anomaly detection, and feature extraction**.
- To encourage meaningful representations, constraints such as **small hidden layers, regularization (L1/L2), denoising, and sparse activations** are applied.
- Common types of autoencoders include:
  5. **Denoising Autoencoder** – Learns to reconstruct clean data from noisy input.
  6. **Sparse Autoencoder** – Activates only a few neurons at a time to enforce sparsity.
  7. **Variational Autoencoder (VAE)** – Learns probabilistic latent representations and can generate new data.
  8. **Convolutional Autoencoder** – Uses CNNs for image data processing.

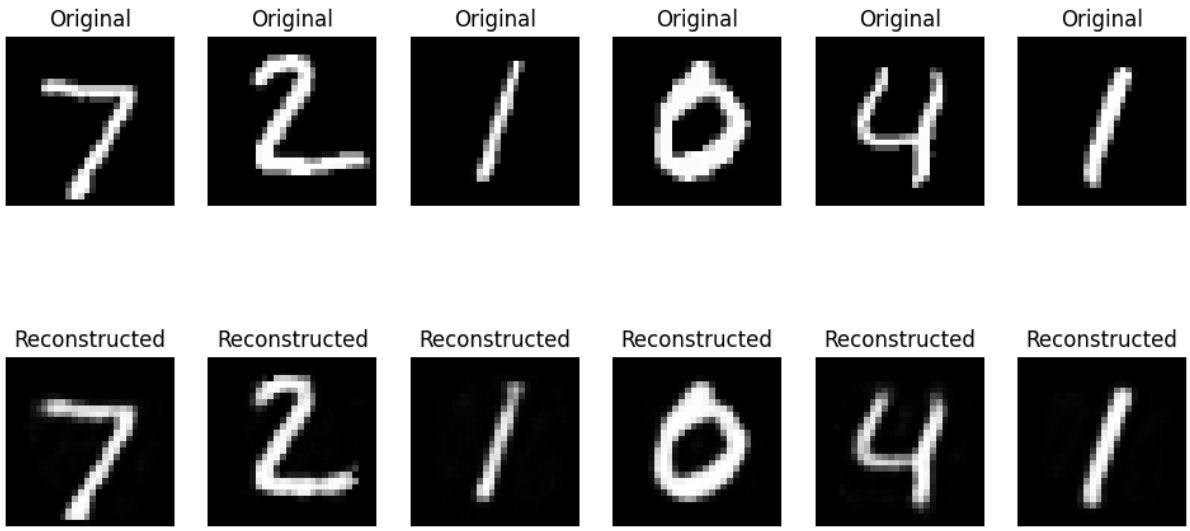
Code:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, losses
from tensorflow.keras.models import Model
from keras.datasets import mnist

(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = np.reshape(x_train, (len(x_train), 28, 28, 1))
x_test = np.reshape(x_test, (len(x_test), 28, 28, 1))
class SimpleAutoencoder(Model):
    def __init__(self, latent_dimensions):
        super(SimpleAutoencoder, self).__init__()
        self.encoder = tf.keras.Sequential([
            layers.Input(shape=(28, 28, 1)),
            layers.Flatten(),
            layers.Dense(latent_dimensions, activation='relu'),
        ])

        self.decoder = tf.keras.Sequential([
            layers.Dense(28 * 28, activation='sigmoid'),
            layers.Reshape((28, 28, 1))
        ])

    def call(self, input_data):
```

```
        encoded = self.encoder(input_data)
        decoded = self.decoder(encoded)
        return decoded

latent_dimensions = 64
autoencoder = SimpleAutoencoder(latent_dimensions)
autoencoder.compile(optimizer='adam', loss=losses.MeanSquaredError())

autoencoder.fit(x_train, x_train,
                epochs=10,
                batch_size=256,
                shuffle=True,
                validation_data=(x_test, x_test))

encoded_imgs = autoencoder.encoder(x_test).numpy()
decoded_imgs = autoencoder.decoder(encoded_imgs).numpy()

n = 6
plt.figure(figsize=(12, 6))
for i in range(n):
    ax = plt.subplot(2, n, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap='gray')
    plt.title("Original")
    plt.axis('off')

    ax = plt.subplot(2, n, i + 1 + n)
    plt.imshow(decoded_imgs[i].reshape(28, 28), cmap='gray')
    plt.title("Reconstructed")
    plt.axis('off')

plt.show()
```

Output:

```
Epoch 1/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 10ms/step - loss: 0.0950 - val_loss: 0.0314
Epoch 2/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0282 - val_loss: 0.0197
Epoch 3/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0185 - val_loss: 0.0144
Epoch 4/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - loss: 0.0137 - val_loss: 0.0110
Epoch 5/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.0108 - val_loss: 0.0090
Epoch 6/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.0088 - val_loss: 0.0075
Epoch 7/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0074 - val_loss: 0.0065
Epoch 8/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 8ms/step - loss: 0.0065 - val_loss: 0.0058
Epoch 9/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.0058 - val_loss: 0.0053
Epoch 10/10
235/235 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.0053 - val_loss: 0.0050
<keras.src.callbacks.history.History at 0x7dea1bcc0f20>
```



Learnings:

Use Cases: Autoencoders are widely used for compression, denoising, anomaly detection, feature learning, and data generation, especially when learning efficient representations is important.

11. Dimensionality Reduction: Compresses high-dimensional data into a smaller latent representation while preserving important information. Often used as a nonlinear alternative to PCA.
12. Noise Removal (Denoising): Learns to reconstruct clean data from noisy inputs, improving data quality in images, audio, and signals.
13. Anomaly Detection: Detects unusual patterns by identifying inputs with high reconstruction error compared to normal training data.
14. Feature Extraction: Learns meaningful latent representations that can be used as input features for classification, clustering, or other ML tasks.
15. Data Compression: Encodes data into compact representations for efficient storage and transmission.
16. Image Reconstruction and Enhancement: Rebuilds or improves images, supporting tasks like inpainting, super-resolution, and image restoration.
17. Generative Modeling (e.g., Variational Autoencoders): Generates new synthetic data samples by learning the underlying data distribution.
18. Recommendation Systems: Learns latent user–item relationships to provide personalized recommendations.
19. Missing Data Imputation: Fills in missing or incomplete values by reconstructing the most probable original data.
20. Unsupervised Pretraining of Deep Networks: Initializes neural networks with learned representations, improving performance when labeled data is limited.

Working:

- Encoder compresses the input data into smaller feature representations.
- Bottleneck (Latent Space) stores the most compact and important representation of the data.
- Decoder reconstructs the original data from this compressed form.
- The model is trained by minimizing reconstruction error, so the output is as close as possible to the input.

Advantages and Benefits:

- Efficiently captures important features from data.
- Useful for noise reduction (denoising).
- Effective in anomaly detection.
- Learn compact and meaningful data representations.

Disadvantages and Limitations:

- May memorize training data instead of learning general patterns (overfitting).
- Reconstructed output can be blurry or imperfect.
- Require large datasets for good performance.
- Needs careful parameter tuning (latent size, learning rate, architecture).
- Performance depends heavily on model design and training quality.

Practical No: 09

Aim: Demonstrate recurrent neural network that learns to perform sequence analysis for stock price. (google stock price)

Implementing deep neural network for performing binary classification task.

Description:

A **Recurrent Neural Network (RNN)** is a type of deep learning architecture designed to work with **sequential/time series data** such as stock prices. RNNs maintain an internal state ("memory") so they can capture temporal patterns over time steps — making them well suited for tasks like forecasting future movements in stock prices based on past history.

Unlike CNNs (image-focused), RNNs (especially LSTM/GRU variants) handle sequences where order and dependency across time matter.

Key components of an RNN for stock price sequence classification:

• **Sequence Windowing:** Input sequences of past n days' prices
• **RNN/LSTM Layers:** Capture long-term dependencies in price history
• **Dense Output Layer:** Performs binary classification (Up vs Down)
• **Activation Functions:** ReLU for hidden layers, sigmoid for final binary output

The workflow includes:

1. **Data Preprocessing:**
   - Load historical Google stock prices
   - Normalize price values
   - Form sequential windows (e.g., 60 days)
   - Label each sequence based on next-day movement (up/down)
   - Split into training and testing sets
2. **Model Architecture:**
   - RNN using LSTM layers to learn temporal context from sequences
   - Binary output layer with sigmoid activation
3. **Training the Model:**
   - Train using binary cross-entropy loss and Adam optimizer
   - Monitor accuracy and validation performance
4. **Evaluation:**
   - Evaluate model accuracy and loss on test data
5. **Prediction:**
   - Use the trained model to predict the next-day movement of Google stock

This RNN approach leverages sequential learning and is commonly used in financial time series analysis.

Code: -

```
pip install numpy pandas matplotlib scikit-learn tensorflow yfinance

import yfinance as yf
import pandas as pd

# Download Google stock data
data = yf.download("GOOGL", start="2015-01-01", end="2023-01-01")

# Use closing price
data = data[['Close']]
data.head()

import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Normalize data
scaler = MinMaxScaler(feature_range=(0,1))
scaled_data = scaler.fit_transform(data)

# Create sequences
def create_sequences(dataset, time_step=60):
    X, y = [], []
    for i in range(time_step, len(dataset)):
        X.append(dataset[i-time_step:i, 0])
        y.append(dataset[i, 0])
    return np.array(X), np.array(y)
```

```
time_step = 60
X, y = create_sequences(scaled_data, time_step)

# Reshape for RNN [samples, time steps, features]
X = X.reshape(X.shape[0], X.shape[1], 1)

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, LSTM, Dropout

model = Sequential()

model.add(LSTM(units=50, return_sequences=True, input_shape=(X.shape[1], 1)))
model.add(Dropout(0.2))

model.add(LSTM(units=50))
model.add(Dropout(0.2))
```

```
Epoch 1/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 7s 60ms/step - loss: 0.0437
Epoch 2/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 62ms/step - loss: 0.0031
Epoch 3/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 59ms/step - loss: 0.0022
Epoch 4/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 55ms/step - loss: 0.0026
Epoch 5/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 58ms/step - loss: 0.0022
Epoch 6/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 55ms/step - loss: 0.0019
Epoch 7/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 58ms/step - loss: 0.0018
Epoch 8/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 58ms/step - loss: 0.0021
Epoch 9/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 48ms/step - loss: 0.0021
Epoch 10/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 55ms/step - loss: 0.0020
Epoch 11/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 2s 36ms/step - loss: 0.0015
Epoch 12/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 52ms/step - loss: 0.0016
Epoch 13/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 52ms/step - loss: 0.0015
Epoch 14/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 49ms/step - loss: 0.0019
Epoch 15/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 55ms/step - loss: 0.0019
Epoch 16/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 54ms/step - loss: 0.0015
Epoch 17/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 3s 54ms/step - loss: 0.0018
Epoch 18/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 61ms/step - loss: 0.0018
Epoch 19/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 61ms/step - loss: 0.0014
Epoch 20/20
62/62 ━━━━━━━━━━━━━━━━━━━━ 4s 62ms/step - loss: 0.0015
<keras.src.callbacks.history.History at 0x2b38d1caf10>
```

Ticker	GOOGL
Date	
2015-01-02	26.278944
2015-01-05	25.778225
2015-01-06	25.142036
2015-01-07	25.068094
2015-01-08	25.155434

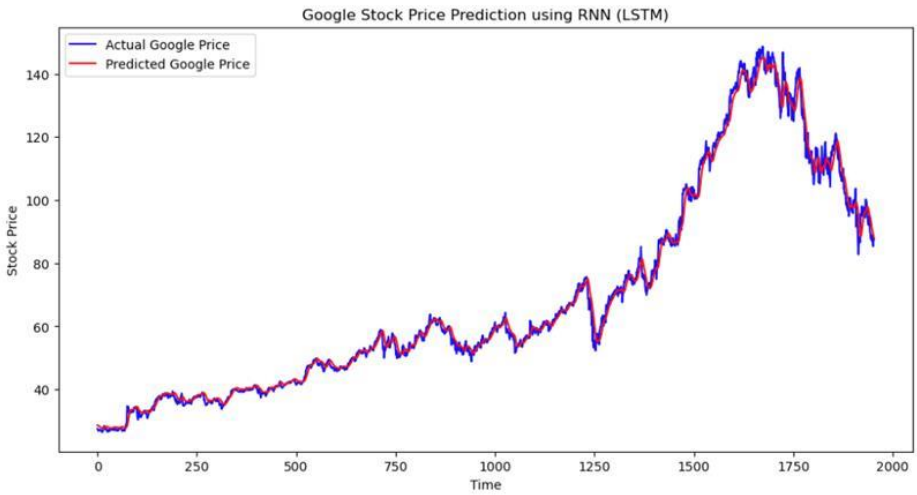
Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 60, 50)	10,400
dropout (Dropout)	(None, 60, 50)	0
lstm_1 (LSTM)	(None, 50)	20,200
dropout_1 (Dropout)	(None, 50)	0
dense (Dense)	(None, 1)	51

Total params: 30,651 (119.73 KB)

Trainable params: 30,651 (119.73 KB)

Non-trainable params: 0 (0.00 B)



Learnings:

In this project, a **Recurrent Neural Network (RNN) with LSTM layers** was successfully implemented to perform sequence analysis on Google stock price data for binary classification.

- The model learned temporal patterns from past prices and used them to predict the direction of future stock movement.
- The accuracy and loss graphs demonstrate how well the model learned over training and validation data.
- Sample predictions show the model's ability to forecast short-term trends from sequential price history.

Overall, this project highlights the strength of RNNs in handling sequential time series data and their application in financial forecasting.

Practical No: 10

Aim: Applying Generative Adversarial Networks for image generation and unsupervised tasks.

Description:

Generative Adversarial Networks (GANs) are deep learning models introduced by Ian Goodfellow in 2014. GANs consist of two neural networks:

1. **Generator (G):**
   - Takes random noise as input.
   - Generates synthetic (fake) images.
   - Tries to fool the discriminator.
2. **Discriminator (D):**
   - Takes real and generated images as input.
   - Classifies whether an image is real or fake.
   - Tries to correctly distinguish real from fake images.

Both networks are trained simultaneously in a competitive process called adversarial training.

GAN for Image Generation:

- Learns the distribution of real images.
- Generates realistic new images from random noise.

GAN for Unsupervised Learning:

- Does not require labeled data.
- Learns meaningful feature representations.
- Useful for clustering, anomaly detection, and data augmentation.

In this experiment, we train a simple GAN to generate handwritten digit images similar to MNIST.

Code: -

```
import numpy as np

import tensorflow as tf
from tensorflow.keras import layers

# 1. Load MNIST Dataset
 (X_train, _), (_, _) = tf.keras.datasets.mnist.load_data()

# Normalize images to [-1, 1]
X_train = (X_train - 127.5) / 127.5
X_train = X_train.reshape(-1, 784).astype("float32")

BUFFER_SIZE = 60000
BATCH_SIZE = 128
EPOCHS = 20
noise_dim = 100
```

```
train_dataset = tf.data.Dataset.from_tensor_slices(X_train)\
    .shuffle(BUFFER_SIZE)\
    .batch(BATCH_SIZE)

# -----------------------------
# 2. Build Generator
# -----------------------------
def build_generator():
    model = tf.keras.Sequential([
        layers.Dense(256, activation='relu', input_shape=(noise_dim,)),
        layers.Dense(512, activation='relu'),
        layers.Dense(784, activation='tanh')
    ])
    return model

# 3. Build Discriminator
def build_discriminator():
    model = tf.keras.Sequential([
        layers.Dense(512, activation='relu', input_shape=(784,)),
        layers.Dense(256, activation='relu'),
        layers.Dense(1, activation='sigmoid')
    ])
    return model

generator = build_generator()
discriminator = build_discriminator()

# -----------------------------
# 4. Define Loss & Optimizers
# -----------------------------
cross_entropy = tf.keras.losses.BinaryCrossentropy()
generator_optimizer = tf.keras.optimizers.Adam(0.0002)
discriminator_optimizer = tf.keras.optimizers.Adam(0.0002)

# 5. Training Step
@tf.function
def train_step(images):
    noise = tf.random.normal([tf.shape(images)[0], noise_dim])

    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        generated_images = generator(noise, training=True)

        real_output = discriminator(images, training=True)
        fake_output = discriminator(generated_images, training=True)

        gen_loss = cross_entropy(tf.ones_like(fake_output), fake_output)

        real_loss = cross_entropy(tf.ones_like(real_output), real_output)
        fake_loss = cross_entropy(tf.zeros_like(fake_output), fake_output)
        disc_loss = real_loss + fake_loss
```

```
    gradients_of_generator = gen_tape.gradient(gen_loss, generator.trainable_variables)
    gradients_of_discriminator = disc_tape.gradient(disc_loss,
discriminator.trainable_variables)

    generator_optimizer.apply_gradients(zip(gradients_of_generator,
generator.trainable_variables))
    discriminator_optimizer.apply_gradients(zip(gradients_of_discriminator,
discriminator.trainable_variables))

    return gen_loss, disc_loss

# 6. Training Loop
for epoch in range(EPOCHS):
    for image_batch in train_dataset:
        g_loss, d_loss = train_step(image_batch)

    print(f"Epoch {epoch+1}/{EPOCHS} | Generator Loss: {g_loss:.4f} | Discriminator Loss:
{d_loss:.4f}")

# 7. Generate Sample Images
print("\nGenerating Sample Images...")

noise = tf.random.normal([5, noise_dim])
generated_images = generator(noise, training=False)

# Reshape to 28x28 images
generated_images = generated_images.numpy().reshape(5, 28, 28)

for i in range(5):
    print(f"\nGenerated Image {i+1} Pixel Values (first 5x5 block):")
    print(generated_images[i][:5, :5])

print("\nGAN Training Completed Successfully.")
```

Output: -

```
Epoch 1/20 | Generator Loss: 6.2632 | Discriminator Loss: 0.0065
Epoch 2/20 | Generator Loss: 2.6012 | Discriminator Loss: 0.1817
Epoch 3/20 | Generator Loss: 3.5489 | Discriminator Loss: 0.0930
Epoch 4/20 | Generator Loss: 4.8436 | Discriminator Loss: 0.0236
Epoch 5/20 | Generator Loss: 6.1143 | Discriminator Loss: 0.0458
Epoch 6/20 | Generator Loss: 4.2466 | Discriminator Loss: 0.0474
Epoch 7/20 | Generator Loss: 3.9252 | Discriminator Loss: 0.0628
Epoch 8/20 | Generator Loss: 4.1882 | Discriminator Loss: 0.2778
Epoch 9/20 | Generator Loss: 4.6071 | Discriminator Loss: 0.0332
Epoch 10/20 | Generator Loss: 8.8365 | Discriminator Loss: 0.1937
Epoch 11/20 | Generator Loss: 6.2728 | Discriminator Loss: 0.0360
Epoch 12/20 | Generator Loss: 5.9778 | Discriminator Loss: 0.0700
Epoch 13/20 | Generator Loss: 5.7012 | Discriminator Loss: 0.1506
Epoch 14/20 | Generator Loss: 5.0942 | Discriminator Loss: 0.0526
Epoch 15/20 | Generator Loss: 5.4851 | Discriminator Loss: 0.0439
Epoch 16/20 | Generator Loss: 5.5778 | Discriminator Loss: 0.1321
Epoch 17/20 | Generator Loss: 4.6448 | Discriminator Loss: 0.0500
Epoch 18/20 | Generator Loss: 5.3209 | Discriminator Loss: 0.2069
Epoch 19/20 | Generator Loss: 4.6415 | Discriminator Loss: 0.2717
Epoch 20/20 | Generator Loss: 5.4251 | Discriminator Loss: 0.0786

Generating Sample Images...

Generated Image 1 Pixel Values (first 5x5 block):
[[-0.9999985  -0.9999987  -0.9999901  -0.9999974  -0.99813485]
 [-0.99997574 -0.9999901  -0.99999785 -0.9998766  -0.9999932 ]
 [-0.99990946 -0.99999744 -0.9999938  -0.9999103  -0.99344856]
 [-0.99998176 -0.9999905  -0.999999   -0.99994665 -0.9996676 ]
 [-0.9999971  -0.9999404  -0.9999792  -0.9999292  -0.9999896 ]]

Generated Image 2 Pixel Values (first 5x5 block):
[[-0.9947898  -0.9897256  -0.9895233  -0.95068306 -0.7126287 ]
 [-0.9481801  -0.99676025 -0.9645578  -0.96312433 -0.9981143 ]
 [-0.908688   -0.96617144 -0.9342558  -0.9898357  -0.8334136 ]
 [-0.9582961  -0.9749638  -0.94191957 -0.9832581  -0.98664665]
 [-0.93603677 -0.99809796 -0.8199596  -0.98549396 -0.98789966]]

Generated Image 3 Pixel Values (first 5x5 block):
[[-0.9945087  -0.997868   -0.99777955 -0.9629546  -0.93782574]
 [-0.9968725  -0.99980974 -0.99676585 -0.9998207  -0.99252945]
 [-0.871863   -0.9987204  -0.9918154  -0.9909827  -0.26148716]
 [-0.9976287  -0.9980128  -0.9997715  -0.9984102  -0.99964625]
 [-0.9962424  -0.99363655 -0.98893636 -0.83386403 -0.9980545 ]]

Generated Image 4 Pixel Values (first 5x5 block):
[[-0.94899887 -0.93946695 -0.9493244  -0.86228526 -0.59471184]
 [-0.9785614  -0.9927304  -0.9490351  -0.9837879  -0.9497332 ]
 [-0.7739927  -0.9497468  -0.8152642  -0.9141584  -0.2836152 ]
 [-0.9411569  -0.96028465 -0.987088   -0.9839252  -0.97248924]
 [-0.9157738  -0.97970456 -0.9121846  -0.89989364 -0.95146954]]

Generated Image 5 Pixel Values (first 5x5 block):
[[-0.97504514 -0.9950628  -0.99318355 -0.95351726 -0.6496854 ]
 [-0.9921129  -0.99973434 -0.9976149  -0.99683595 -0.9988526 ]
 [-0.63776386 -0.9952073  -0.9309477  -0.9872223  -0.12858039]
 [-0.99452406 -0.98653305 -0.99882555 -0.9975533  -0.9991051 ]
 [-0.9866552  -0.9976479  -0.91831964 -0.9729559  -0.9973938 ]]

GAN Training Completed Successfully.
```

Learnings:

- **GAN consists of two competing networks: Generator and Discriminator.**
  The Generator creates fake samples from random noise, while the Discriminator evaluates whether the samples are real or fake. Their adversarial competition improves both networks over time.
- **GAN does not require labeled data → fully unsupervised learning.**
  GANs learn the underlying data distribution without class labels. This makes them powerful for tasks where labeled data is scarce or expensive.

- **Generator learns to model the data distribution from random noise.**
  By mapping random vectors to realistic outputs, the Generator approximates the true probability distribution of the dataset.